

CSD361 — Lab Assignment 1

Chicago Taxi Trips Dataset (2024) — Fare Prediction

Part I — Dataset Exploration: 1. Observations & attributes · 2. Attribute types · 3. Missing values · 4. Duplicates · 5. Descriptive statistics · 6. Distribution plots

Part II — The Prediction Problem: Linear Regression for FARE — Feature engineering and selection

Part III — Build Three Regression Models

Part IV — Effect of Scaling on Gradient Descent — IV.1-2 Same learning rate · IV.3 Learning-rate sweep · IV.4 Discussion

Part V — How Do Outliers Affect Normalization? — V.1 IQR outliers · V.2 Dataset A vs B · V.3 Discussion

Overall Conclusions

Code file: <https://github.com/Tanvi-Pandey/chicago-taxi-fare-linear-regression>

Dataset: Chicago Taxi Trips — 2024 (Taxi_Trips_-_2024_20240408.csv), 865,247 trips, 23 attributes.

1 & 2. Number of observations/attributes, and attribute types

```
# 1. Number of observations and attributes
print(f"Number of observations (rows): {df.shape[0]}")
print(f"Number of attributes (columns): {df.shape[1]}")
print()
print(df.dtypes)
```

```
Number of observations (rows): 865247
Number of attributes (columns): 23
```

```
Trip ID                str
Taxi ID                str
Trip Start Timestamp   str
Trip End Timestamp     str
Trip Seconds           float64
Trip Miles             float64
Pickup Census Tract    float64
Dropoff Census Tract   float64
Pickup Community Area  float64
Dropoff Community Area float64
Fare                  float64
Tips                 float64
Tolls                float64
Extras               float64
Trip Total           float64
Payment Type         str
Company              str
Pickup Centroid Latitude float64
Pickup Centroid Longitude float64
Pickup Centroid Location str
Dropoff Centroid Latitude float64
Dropoff Centroid Longitude float64
Dropoff Centroid Location str
dtype: object
```

Code and output for Part I.1–2: shape and dtypes of the dataset

Type	Attributes
------	------------

Numerical	Trip Seconds, Trip Miles, Fare, Tips, Tolls, Extras, Trip Total, Pickup/Dropoff Centroid Latitude & Longitude
Categorical	Payment Type, Company, Pickup Community Area, Dropoff Community Area
Identifier	Trip ID, Taxi ID, Pickup/Dropoff Census Tract, Pickup/Dropoff Centroid Location (WKT string)
Timestamp	Trip Start Timestamp, Trip End Timestamp

(Community Area is a location code, not a real number — treated as categorical.) **3.**

Missing values per attribute

Attribute	Missing Count	Missing %
Dropoff Census Tract	551393	63.73
Pickup Census Tract	537996	62.18
Dropoff Community Area	83853	9.69
Dropoff Centroid Location	79675	9.21
Dropoff Centroid Latitude	79675	9.21
Dropoff Centroid Longitude	79675	9.21
Pickup Community Area	23195	2.68
Pickup Centroid Latitude	22966	2.65
Pickup Centroid Location	22966	2.65
Pickup Centroid Longitude	22966	2.65
Fare	2045	0.24
Extras	2045	0.24
Tolls	2045	0.24
Trip Total	2045	0.24
Tips	2045	0.24
Trip Seconds	169	0.02
Trip End Timestamp	8	0.00
Trip Miles	6	0.00
Taxi ID	1	0.00

4. Duplicate records

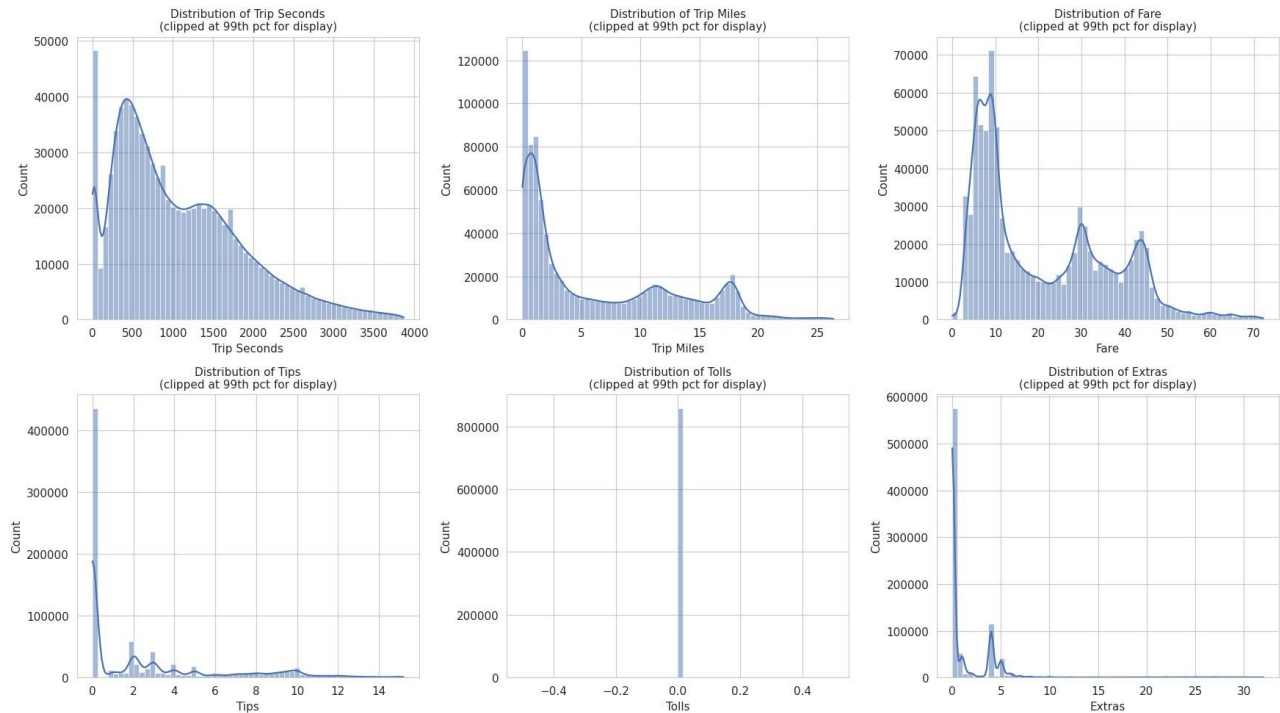
Fully duplicated rows: 0. Duplicate Trip IDs: 0. No duplicate records were found in this dataset.

5. Descriptive statistics

	count	mean	std	min	25%	50%	75%	max	skew	missing
Trip Seconds	865078.0	1150.53	1584.24	0.0	459.00	900.00	1591.00	86340.00	24.58	169
Trip Miles	865241.0	6.56	8.28	0.0	0.91	3.07	11.71	3093.47	75.07	6
Fare	863202.0	22.06	25.15	0.0	8.00	15.75	33.75	9999.75	159.40	2045
Tips	863202.0	2.76	4.13	0.0	0.00	0.00	4.00	200.00	3.06	2045
Tolls	863202.0	0.06	12.96	0.0	0.00	0.00	0.00	4444.44	323.05	2045
Extras	863202.0	2.15	19.32	0.0	0.00	0.00	3.00	5051.10	198.06	2045

All six columns are skewed to the right (most values are small, but a few are very large). A small number of extreme values pull the maximum far above the 75th percentile. This is why we look at outliers more closely in Part V.

6. Distribution plots



Distributions of the six key numerical variables (clipped at the 99th percentile for readability)

Part II — The Prediction Problem: Linear Regression for FARE

Feature engineering and selection

Feature engineering and selection

```
[22]: work = df.copy()
work['Trip Start Timestamp'] = pd.to_datetime(work['Trip Start Timestamp'], errors='coerce')

core_cols = ['Fare', 'Trip Seconds', 'Trip Miles', 'Pickup Community Area',
             'Dropoff Community Area', 'Trip Start Timestamp']
work = work.dropna(subset=core_cols)

work = work[(work['Trip Seconds'] > 0) & (work['Trip Miles'] > 0) & (work['Fare'] > 0)]

work = work[(work['Trip Seconds'] <= 4*3600) & (work['Fare'] <= 300) & (work['Trip Miles'] <= 100)]

work['hour'] = work['Trip Start Timestamp'].dt.hour
work['day_of_week'] = work['Trip Start Timestamp'].dt.dayofweek # 0 = Monday
work['month'] = work['Trip Start Timestamp'].dt.month
work['is_weekend'] = (work['day_of_week'] >= 5).astype(int)

FEATURES = ['Trip Seconds', 'Trip Miles', 'Pickup Community Area',
            'Dropoff Community Area', 'hour', 'day_of_week', 'month', 'is_weekend']
TARGET = 'Fare'

model_df = work[FEATURES + [TARGET]].dropna().reset_index(drop=True)
print(f"Rows after cleaning: {len(model_df)}")
print(f"Selected features: {FEATURES}")

C:\Users\Tanvi\AppData\Local\Temp\ipykernel_10128\3316444119.py:2: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
  work['Trip Start Timestamp'] = pd.to_datetime(work['Trip Start Timestamp'], errors='coerce')
Rows after cleaning: 696310
Selected features: ['Trip Seconds', 'Trip Miles', 'Pickup Community Area', 'Dropoff Community Area', 'hour', 'day_of_week', 'month', 'is_weekend']
```

Cleaning, temporal feature extraction, and final feature selection, with output

Why these features were chosen

- **Trip Seconds & Trip Miles** — fares are mainly based on how long and how far the trip is, so these are the two most useful features.
- **Pickup/Dropoff Community Area** — the pickup/drop-off zone affects price too (for example, airport trips or highway routes), even for a similar distance.
- **hour, day_of_week, month, is_weekend** (taken from the timestamp) — capture things like rushhour traffic and demand changes by time of day, day, or season.
- **Excluded:** Tips, Tolls, Extras, and Trip Total were left out because they aren't known before the ride, and Trip Total is literally calculated from Fare — using it would be cheating (data leakage). Company, Taxi ID, and Trip ID are just labels with no predictive value. The census tract columns were dropped for having too many missing values (over 60%).

For Parts III–V, a random sample of 60,000 rows (out of the 696,310 cleaned rows) was used, mainly to keep the from-scratch gradient descent code in Part IV fast enough to run.

Part III — Build Three Regression Models

```

results = {}

# Model 1
lr1 = LinearRegression()
lr1.fit(X_train, y_train)
pred1 = lr1.predict(X_test)
mae1, mse1 = mean_absolute_error(y_test, pred1), mean_squared_error(y_test, pred1)
results['Original'] = (mae1, mse1)
print(f"Model 1 (Original):      MAE={mae1:.4f}  MSE={mse1:.4f}")

# Model 2
scaler_mm = MinMaxScaler()
X_train_mm, X_test_mm = scaler_mm.fit_transform(X_train), scaler_mm.transform(X_test)
lr2 = LinearRegression()
lr2.fit(X_train_mm, y_train)
pred2 = lr2.predict(X_test_mm)
mae2, mse2 = mean_absolute_error(y_test, pred2), mean_squared_error(y_test, pred2)
results['Min-Max'] = (mae2, mse2)
print(f"Model 2 (Min-Max):      MAE={mae2:.4f}  MSE={mse2:.4f}")

# Model 3
scaler_std = StandardScaler()
X_train_std, X_test_std = scaler_std.fit_transform(X_train), scaler_std.transform(X_test)
lr3 = LinearRegression()
lr3.fit(X_train_std, y_train)
pred3 = lr3.predict(X_test_std)
mae3, mse3 = mean_absolute_error(y_test, pred3), mean_squared_error(y_test, pred3)
results['Standardized'] = (mae3, mse3)
print(f"Model 3 (Standardized): MAE={mae3:.4f}  MSE={mse3:.4f}")

print()

```

Fitting the Original, Min-Max, and Standardized linear regression models

Model	MAE	MSE
Original	2.6938	28.4119
Min-Max	2.6938	28.4119
Standardized	2.6938	28.4119

3. Does normalization change the MAE / MSE?

No, the numbers come out exactly the same. Linear regression (solved directly, not with gradient descent) always finds the best possible fit no matter how the features are scaled — scaling just changes the size of the coefficients, not the final predictions.

4. Does performance stay the same for all 3 models? Why?

Yes. Min-Max and Standardization are just simple, reversible rescalings of the same data. Since linear regression can "undo" any such rescaling through its coefficients, the predictions end up identical either way.

5. Which model would you recommend, and why?

Since all three give the same accuracy here, it doesn't matter for this specific case. But in general, the Standardized version is still the better choice, because: (a) its coefficients are easier to compare and interpret, and (b) it works much better with methods that actually are sensitive to scale, like gradient descent or regularization — which is exactly what we see in Part IV next.

Part IV — Effect of Scaling on Gradient Descent

Here we write our own gradient descent using NumPy (instead of sklearn) and compare how it behaves on the Original, Min-Max, and Standardized feature sets.

IV.1-2 — Same learning rate for all three models

IV.1-2

```
[6]: LR_COMMON = 0.001
N_ITERS = 3000
histories = {}
for name, Xd in datasets.items():
    w, hist = gradient_descent(Xd, y_train_col, lr=LR_COMMON, n_iters=N_ITERS)
    histories[name] = hist
    finite_hist = hist[np.isfinite(hist)]
    if len(finite_hist) < N_ITERS:
        print(f"{name:15s}: DIVERGED at iteration {len(finite_hist)} (lr={LR_COMMON})")
    else:
        print(f"{name:15s}: Final MSE = {hist[-1]:.4f}")

plt.figure(figsize=(9,6))
for name, hist in histories.items():
    plt.plot(np.clip(hist, 0, 1e5), label=name, linewidth=2)
plt.xlabel('Iteration'); plt.ylabel('MSE (training)'); plt.yscale('log')
plt.title(f'MSE vs Iteration - same learning rate (lr={LR_COMMON}) for all 3 models')
plt.legend(); plt.tight_layout(); plt.show()

Original      : DIVERGED at iteration 3 (lr=0.001)
Min-Max       : Final MSE = 143.5992
Standardized   : Final MSE = 26.9607
```

Gradient descent implementation and run at lr=0.001 for all three feature sets

Model	Result
Original	DIVERGED at iteration 3 (lr=0.001)
Min-Max	Final MSE = 143.5992
Standardized	Final MSE = 26.9607

a. Iterations needed to converge

Standardized features converge the fastest by far. Min-Max features are much slower — even after 3000 iterations, they still haven't fully settled. Original (unscaled) features **blow up almost instantly** at this learning rate, because their raw values (like Trip Seconds going up to ~14,000) are too large for this step size.

b. Final MSE

Standardized ends up at about 27, basically matching the exact answer (28.41) from Part III. Min-Max is stuck around 144 (still not done converging). Original has completely diverged (MSE → infinity).

c. Sensitivity to learning rate

Original features are extremely fragile — you'd need a tiny, tiny learning rate to keep them stable. Standardized features are the most forgiving and work well across a wide range of learning rates.

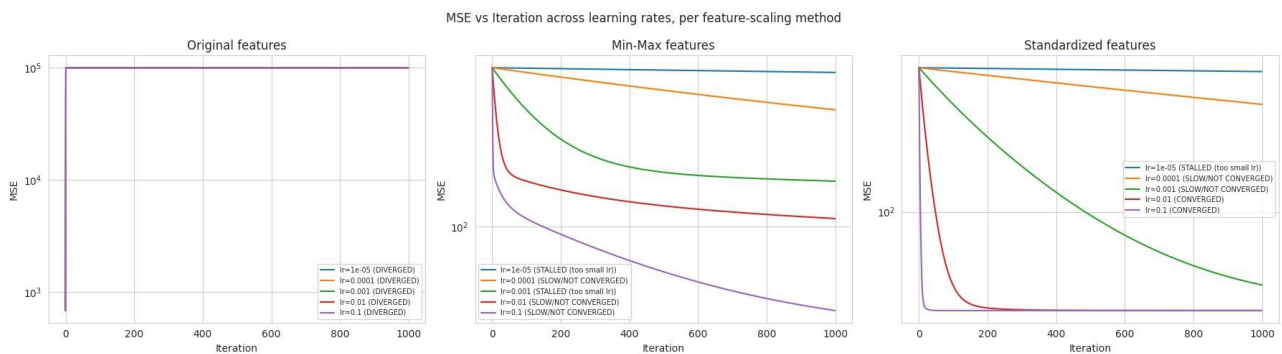
IV.3 — Learning-rate sweep: 0.00001, 0.0001, 0.001, 0.01, 0.1

```
#IV.3
learning_rates = [0.00001, 0.0001, 0.001, 0.01, 0.1]
N_ITERS_SWEEP = 1000
sweep_results, sweep_final_mse = {name: {} for name in datasets}, {name: {} for name in datasets}

fig, axes = plt.subplots(1, 3, figsize=(18, 5))
for ax, (name, Xd) in zip(axes, datasets.items()):
    hist = {lr: gradient_descent(Xd, y_train_col, lr=lr, n_iters=N_ITERS_SWEEP)[1] for lr in learning_rates}
    finite_finals = [h[-1] for h in hist.values() if np.isfinite(h[-1])]
    best_final = min(finite_finals) if finite_finals else np.inf
    for lr in learning_rates:
        hist = hist[lr]
        finite_hist = hist[np.isfinite(hist)]
        if len(finite_hist) < N_ITERS_SWEEP:
            status = 'DIVERGED'
        else:
            rel_change = abs(hist[-1] - hist[-min(50, len(hist))]) / (hist[-min(50, len(hist))] + 1e-9)
            near_optimum = hist[-1] <= 1.05 * best_final
            status = 'CONVERGED' if (near_optimum and rel_change < 0.01) else \
                ('STALLED (too small lr)' if rel_change < 0.01 else 'SLOW/NOT CONVERGED')
        sweep_results[name][lr] = status
        sweep_final_mse[name][lr] = hist[-1]
        ax.plot(np.clip(hist, 0, 1e5), label=f'lr={lr} ({status})')
    ax.set_title(f'{name} features'); ax.set_xlabel('Iteration'); ax.set_ylabel('MSE')
    ax.set_yscale('log'); ax.legend(fontsize=7)
plt.suptitle('MSE vs Iteration across learning rates, per feature-scaling method')
plt.tight_layout(); plt.show()

print(pd.DataFrame(sweep_results))
print()
print("Final MSE per (scaling, learning rate):")
print(pd.DataFrame(sweep_final_mse).round(3))
```

Sweep code across five learning rates for all three feature-scaling methods



MSE vs Iteration across learning rates, per feature-scaling method

Learning rate	Original	Min-Max	Standardized
0.00001	DIVERGED	STALLED (too small lr)	STALLED (too small lr)
0.00010	DIVERGED	SLOW/NOT CONVERGED	SLOW/NOT CONVERGED
0.00100	DIVERGED	STALLED (too small lr)	SLOW/NOT CONVERGED
0.01000	DIVERGED	SLOW/NOT CONVERGED	CONVERGED
0.10000	DIVERGED	SLOW/NOT CONVERGED	CONVERGED

Learning rate	Original (final MSE)	Min-Max (final MSE)	Standardized (final MSE)
0.00001	inf	649.445	653.321
0.00010	inf	412.756	421.484
0.00100	inf	173.151	37.594
0.01000	inf	109.937	26.687
0.10000	inf	35.933	26.687

a. Convergence

Only the Standardized features fully converge to the best possible MSE (~27) within 1000 iterations — and this works at both $\text{lr}=0.01$ and $\text{lr}=0.1$, so there's a wide range of safe learning rates to choose from.

b. Slow convergence / stalling

Min-Max features never fully converge in 1000 iterations, no matter which learning rate is used — the smaller $[0,1]$ range just needs more steps to get anywhere. Both scaled versions barely move at the smallest learning rate (0.00001).

c. Divergence

For Original (unscaled) features, every single learning rate tested — from 0.00001 all the way to 0.1 — causes the loss to explode within a few iterations. The raw values are just too large, so even a small step size overshoots badly. A workable learning rate here would need to be far smaller still (roughly 0.0000001 or lower).

IV.4 — Why does normalization change Gradient Descent's behavior?

Each gradient descent step's size depends on how big the feature's values are. When features are on very different scales (like Trip Seconds going up to 14,000 versus `is_weekend` being just 0 or 1), the loss surface ends up shaped like a long, narrow valley instead of a round bowl — steep in some directions, almost flat in others.

A learning rate small enough to be safe in the steep direction is way too small to make progress in the flat direction, so it converges slowly. A learning rate big enough to move fast in the flat direction will overshoot and blow up in the steep direction. Scaling the features (Min-Max or Standardization) makes all the directions roughly the same size, turning that narrow valley into something closer to a round bowl. That's why gradient descent can use one single, larger learning rate and converge much faster once the features are scaled — Standardization gives the roundest, best-behaved bowl of the two.

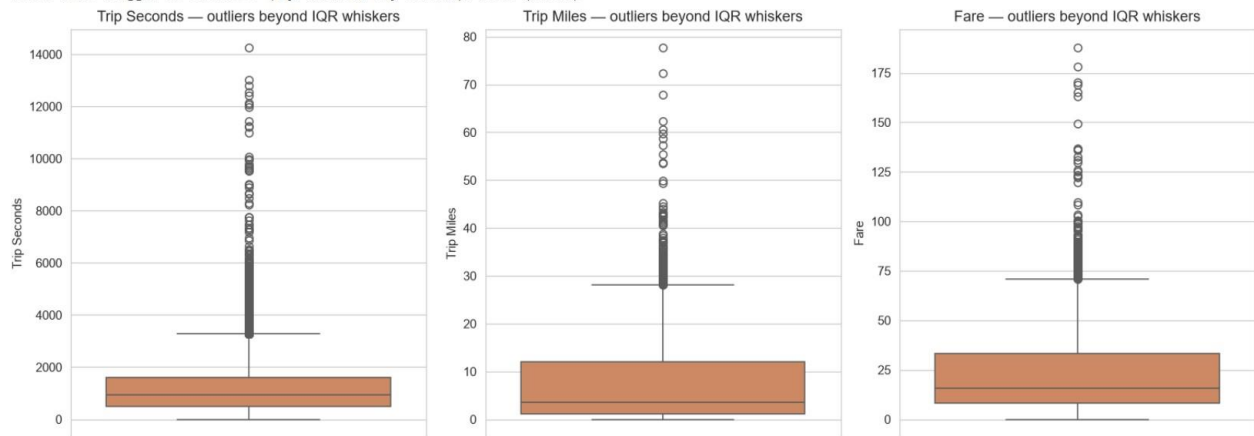
Part V — How Do Outliers Affect Normalization?

V.1 — Identifying outliers (IQR method)

```
for col in ['Trip Seconds', 'Trip Miles', 'Fare']:
    Q1, Q3 = sample_df[col].quantile(0.25), sample_df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower, upper = Q1 - 1.5*IQR, Q3 + 1.5*IQR
    col_outliers = (sample_df[col] < lower) | (sample_df[col] > upper)
```

Trip Seconds : bounds=[-1174.50, 3285.50], outliers=1343 (2.24%)
 Trip Miles : bounds=[-14.97, 28.15], outliers=193 (0.32%)
 Fare : bounds=[-29.25, 70.75], outliers=238 (0.40%)

Total rows flagged as outliers (IQR method, any column): 1532 (2.55%)



Outlier bounds and counts (IQR method), with boxplots beyond the whiskers

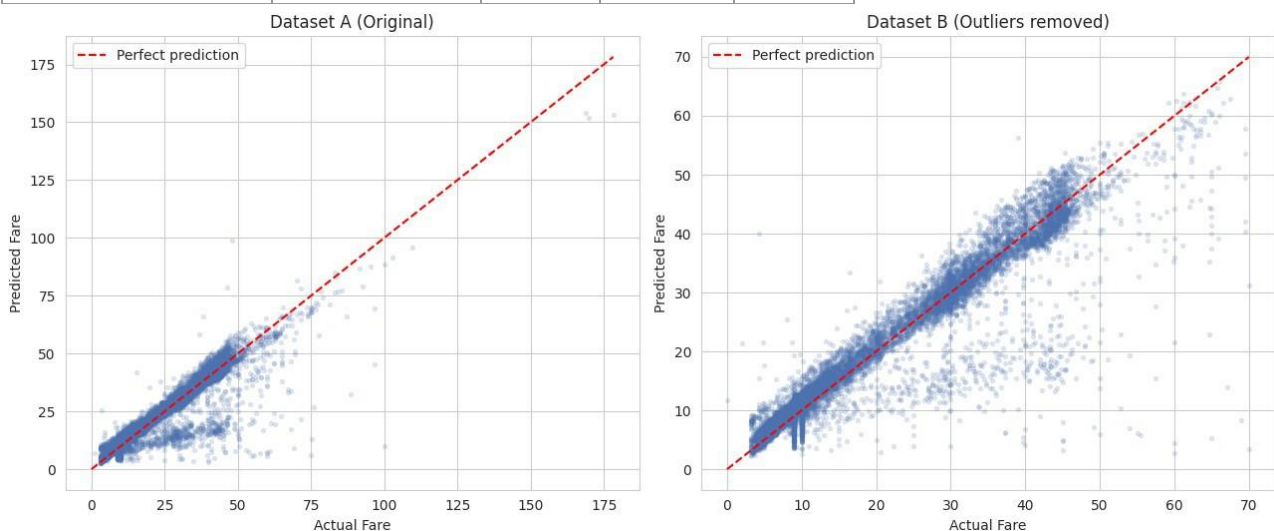
V.2 — Linear Regression on Dataset A (original) vs Dataset B (outliers removed)

```

dataset_A = sample_df.copy()          # Dataset A: original
dataset_B = sample_df[~outlier_mask].copy() # Dataset B: outliers removed

```

Dataset	Rows	MAE	MSE	R ²
A (with outliers)	60000 (100%)	2.6938	28.4119	0.8810
B (outliers removed)	58468 (97.4%)	2.5687	23.3379	0.8875



Predicted vs Actual Fare, Dataset A (with outliers) vs Dataset B (outliers removed)

V.3 — Does removing outliers necessarily improve Linear Regression performance?

Not always — it depends on what you care about.

- Here, MAE improved from 2.69 to 2.57, and MSE improved more, from 28.41 to 23.34 (R² also went up, from 0.881 to 0.888). MSE improves more because it squares the errors, so a few extreme outliers were dragging that number up a lot.

- Some of this improvement is simply because the remaining data is easier to predict (less spread out), not because the model got smarter.
- The risk: if those outliers were real trips (like a genuine long airport ride), the model will now do a worse job predicting fares for exactly those kinds of trips in the future. We traded away accuracy on rare cases for better accuracy on typical ones.
- **Bottom line:** removing outliers made the numbers look better here, but that's not automatically "improvement" — it depends on whether you actually care about predicting those rare, expensive trips well. A gentler option, like capping extreme values instead of deleting them, can sometimes be a better middle ground.

Tanvi Pandey

2410110356

B.Tech CSE-2028